Peerlink

Technical Report (Split Chapters)

C++ Boost.Asio/Beast + React WebSocket

Author: Ansh Prem

February 27, 2026

Contents

1 Introduction **5**

2 How to Compile This Report (No Overleaf Crash) **6**
2.1 Why split into many .tex files? 6
2.2 Compile one chapter at a time 6

3 Backend Code: include/util/Log.hpp **7**
3.1 Purpose 7
3.2 Full Code with Line-by-Line Comments 7

4 Backend Code: src/util/Log.cpp **9**
4.1 Purpose 9
4.2 Full Code with Line-by-Line Comments 9

5 Backend Code: src/main.cpp **12**
5.1 Purpose 12
5.2 Full Code with Line-by-Line Comments 12

6 Backend Code: include/core/CommandParser.hpp **16**
6.1 Purpose 16
6.2 Full Code with Line-by-Line Comments 16

7 Backend Code: src/core/CommandParser.cpp **19**
7.1 Purpose 19
7.2 Full Code with Line-by-Line Comments 19

8 Backend Code: include/core/ChatRoom.hpp **23**
8.1 Purpose 23
8.2 Full Code with Line-by-Line Comments 23

9 Backend Code: src/core/ChatRoom.cpp **26**
9.1 Purpose 26
9.2 Full Code with Line-by-Line Comments 26

10 Backend Code: include/core/RoomManager.hpp **30**
10.1 Purpose 30
10.2 Full Code with Line-by-Line Comments 30

11 Backend Code: src/core/RoomManager.cpp **32**
11.1 Purpose 32
11.2 Full Code with Line-by-Line Comments 32

12 Backend Code: include/net/TcpServer.hpp **35**
12.1 Purpose 35
12.2 Full Code with Line-by-Line Comments 35

13 Backend Code: src/net/TcpServer.cpp **37**
13.1 Purpose 37
13.2 Full Code with Line-by-Line Comments 37

14 Backend Code: include/net/TcpSession.hpp **40**
14.1 Purpose 40
14.2 Full Code with Line-by-Line Comments 40

15 Backend Code: src/net/TcpSession.cpp **43**
15.1 Purpose 43
15.2 Full Code with Line-by-Line Comments 43

16 Backend Code: include/net/WsServer.hpp **50**
16.1 Purpose 50
16.2 Full Code with Line-by-Line Comments 50

17 Backend Code: src/net/WsServer.cpp **52**
17.1 Purpose 52
17.2 Full Code with Line-by-Line Comments 52

18 Backend Code: include/net/WsSession.hpp **55**
18.1 Purpose 55
18.2 Full Code with Line-by-Line Comments 55

19 Backend Code: src/net/WsSession.cpp **59**
19.1 Purpose 59
19.2 Full Code with Line-by-Line Comments 59

20 Frontend Code: web-react/index.html **67**
20.1 Purpose 67
20.2 Full Code with Line-by-Line Comments 67

21 Frontend Code: web-react/package.json **69**
21.1 Purpose 69
21.2 Full Code with Line-by-Line Comments 69

22 Frontend Code: web-react/vite.config.js **71**
22.1 Purpose 71
22.2 Full Code with Line-by-Line Comments 71

23 Frontend Code: web-react/src/main.jsx **73**
23.1 Purpose 73
23.2 Full Code with Line-by-Line Comments 73

24 Frontend Code: web-react/src/App.jsx **74**
24.1 Purpose 74
24.2 Full Code with Line-by-Line Comments 74

25 Frontend Code: web-react/src/styles.css **80**
25.1 Purpose 80
25.2 Full Code with Line-by-Line Comments 80

26 Project Documentation: README.md **83**

27 Deployment Guide: HOW_TO_RUN.txt **84**

28 CMake and Build Notes (Peerlink) **85**
28.1 Linux Build Commands 85
28.2 Frontend Build Commands 85
28.3 Common Fixes 85
28.3.1 Clock Skew Warning in WSL 85
28.3.2 Port Already in Use 85

29 Architecture Deep Dive **87**
29.1 High-Level System Overview 87
29.2 Component Diagram (Safe TikZ) 88
29.3 Asynchronous Flow 88
29.3.1 Accept Phase 88
29.3.2 Message Phase 88
29.4 Event Loop Internals 89
29.5 Concurrency Model 89
29.6 Memory Management Strategy 89
29.7 Why This Architecture is Resume-Heavy 89

30 Performance Analysis **90**
30.1 Thread Model 90
30.2 Expected Scalability 90

30.3 Latency Characteristics . . . 90
30.4 Memory Footprint . . . 91
30.5 Stress Testing Strategy . . . 91
30.6 Structured Logging Advantage . . . 91

31 Conclusion and Future Work **92**
31.1 Project Summary . . . 92
31.2 Key Engineering Learnings . . . 92
31.3 Production-Grade Extensions . . . 92
31.3.1 Authentication . . . 92
31.3.2 Persistent Storage . . . 92
31.3.3 Horizontal Scaling . . . 93
31.3.4 TLS Encryption . . . 93
31.3.5 Message Persistence . . . 93
31.4 Final Remark . . . 93

32 Project Folder Structure (ASCII Only) **94**
32.1 Why ASCII Only . . . 94
32.2 Folder Tree . . . 94
32.3 What Each Folder Contains . . . 95

33 Async Flow and Event Loop Behavior **96**
33.1 Core Idea . . . 96
33.2 Boost.Asio Event Loop . . . 96
33.3 Asynchronous Accept Loop . . . 96
33.4 Flow Diagram (Safe TikZ) . . . 97
33.5 Why This Scales . . . 97

34 Concurrency Model and Memory Management **98**
34.1 Thread Pool Model . . . 98
34.2 Shared State . . . 98
34.3 Session Lifetime Management . . . 98
34.4 Ownership Diagram . . . 99
34.5 Why Strands May Be Used . . . 99

35 Sequence Diagrams (Safe, Page-Fit) **100**
35.1 Join Room Sequence . . . 100

Chapter 1

Introduction

Peerlink is a high-performance asynchronous chat server implemented in C++ using:

- Boost.Asio for async TCP + thread pool event loop
- Boost.Beast for WebSocket support
- Boost.JSON for structured message protocol

The frontend is React-based and uses WebSocket for real-time messaging.

This report is split into many files to avoid Overleaf compilation limits. Each source file has its own chapter and includes:

- Full code
- Line-by-line explanation inside comments

Chapter 2

How to Compile This Report (No Overleaf Crash)

2.1 Why split into many .tex files?

Overleaf can time out or run out of memory if one file contains thousands of lines of code listings. Splitting keeps compilation small and stable.

2.2 Compile one chapter at a time

Open `main.tex` and edit the `\includeonly{...}` list.

Example: compile only Log chapters:

```
\includeonly{
  tex/backend/01_Log_hpp,
  tex/backend/02_Log_cpp
}
```

Then Overleaf compiles fast. After that, enable more chapters gradually.

Chapter 3

Backend Code: include/util/Log.hpp

3.1 Purpose

`util::Log` is a tiny structured logger that prints JSON-like events to stdout. We keep it minimal so:

- no external dependencies
- logs are consistent across TCP + WS + main server
- easy to filter/grade and debug

3.2 Full Code with Line-by-Line Comments

```
// File: include/util/Log.hpp
// Purpose: Minimal logger for structured JSON-like terminal logs.

#pragma once
// Why: Prevent multiple inclusion.

#include <string>
// Why: Log API accepts std::string messages.

namespace util {
// Why: Utility namespace for non-core helpers.

  class Log {
  public:
    // Print an INFO event.
    static void info(const std::string& msg);

    // Print a WARN event.
    static void warn(const std::string& msg);
```

```
    // Print an ERROR event.
    static void error(const std::string& msg);

  private:
    // Internal helper: prints one JSON line.
    // level: "INFO"/"WARN"/"ERROR"
    // msg: message string
    static void log_json(const char* level, const std::string& msg);

    // Helper: returns a short timestamp string.
    static std::string now_ts();

    // Helper: JSON-escape a string so logs stay valid JSON.
    static std::string escape_json(const std::string& s);
  };

} // namespace util
```

s

Chapter 4

Backend Code: src/util/Log.cpp

4.1 Purpose

Implements JSON-safe logging:

- adds timestamp
- escapes quotes/backslashes/newlines to keep JSON valid
- prints one log event per line

4.2 Full Code with Line-by-Line Comments

```
// File: src/util/Log.cpp
// Purpose: Implementation of util::Log.

#include "util/Log.hpp"
// Why: Implement the class declared in the header.

#include <iostream>
// Why: Print to stdout using std::cout.

#include <iomanip>
// Why: time formatting (setw, setfill).

#include <sstream>
// Why: build strings efficiently.

#include <chrono>
// Why: access system_clock time.

#include <ctime>
// Why: convert to tm for readable clock.
```

```
namespace util {

std::string Log::escape_json(const std::string& s) {
  // This function ensures the message becomes JSON-safe.
  // We escape: backslash, double-quote, newline, carriage return, tab.
  std::string out;
  out.reserve(s.size());

  for (char c : s) {
    switch (c) {
      case '\\': out += "\\\\"; break; // \  -> \\
      case '\"': out += "\\\""; break; // "  -> \"
      case '\n': out += "\\n";  break; // newline -> \n
      case '\r': out += "\\r";  break; // CR -> \r
      case '\t': out += "\\t";  break; // tab -> \t
      default:
        // For normal characters, append as-is.
        out += c;
        break;
    }
  }

  return out;
}

std::string Log::now_ts() {
  // Create a timestamp like "2026-02-25 15:20:04".
  using clock = std::chrono::system_clock;

  auto now = clock::now();
  std::time_t t = clock::to_time_t(now);

  // Convert time_t to local time structure.
  std::tm tm{};
#if defined(_WIN32)
  // Windows uses localtime_s.
  localtime_s(&tm, &t);
#else
  // POSIX uses localtime_r.
  localtime_r(&t, &tm);
#endif

  // Format into a string.
  std::ostringstream oss;
  oss << std::put_time(&tm, "%Y-%m-%d %H:%M:%S");
  return oss.str();
```

```
}

void Log::log_json(const char* level, const std::string& msg) {
  // Build a JSON object in one line:
  // {"ts":"...","level":"INFO","msg":"..."}
  // We keep it stable for easy parsing.

  std::string ts = now_ts();
  std::string safe = escape_json(msg);

  std::cout
    << "{\"ts\":\"" << ts
    << "\",\"level\":\"" << level
    << "\",\"msg\":\"" << safe
    << "\"}"
    << std::endl;

  // std::endl flushes; good for real-time debugging.
}

void Log::info(const std::string& msg) {
  // Public API for INFO.
  log_json("INFO", msg);
}

void Log::warn(const std::string& msg) {
  // Public API for WARN.
  log_json("WARN", msg);
}

void Log::error(const std::string& msg) {
  // Public API for ERROR.
  log_json("ERROR", msg);
}

} // namespace util
```

Chapter 5

Backend Code: src/main.cpp

5.1 Purpose

This is the process entry point:

- parses CLI args: TCP port, WS port, thread count
- creates `io_context` and shared `RoomManager`
- starts both TCP and WS servers
- runs the event loop on a thread pool
- handles shutdown signals (Ctrl+C)

5.2 Full Code with Line-by-Line Comments

```
// File: src/main.cpp
// Purpose: Start Peerlink backend (TCP + WebSocket) with thread pool.

#include <boost/asio.hpp>
// Why: io_context, signal_set, threads.

#include <iostream>
// Why: basic error printing.

#include <thread>
// Why: run io_context on multiple threads.

#include <vector>
// Why: store thread objects.

#include <string>
// Why: parse arguments.
```

```
#include <cstdlib>
// Why: std::atoi

#include "core/RoomManager.hpp"
// Why: shared rooms used by all sessions.

#include "net/TcpServer.hpp"
// Why: start TCP acceptor.

#include "net/WsServer.hpp"
// Why: start WebSocket acceptor.

#include "util/Log.hpp"
// Why: structured JSON logs.

struct Args {
  // Bind address for both servers.
  std::string addr = "0.0.0.0";

  // TCP port.
  int tcp = 5555;

  // WS port.
  int ws = 8080;

  // Thread pool size.
  int threads = 4;
};

static Args parse_args(int argc, char** argv) {
  // Parse command line:
  // ./peerlink_server --tcp 5555 --ws 8080 --threads 4 --addr 0.0.0.0
  Args a;

  for (int i = 1; i < argc; ++i) {
    std::string key = argv[i];

    if (key == "--tcp" && i + 1 < argc) {
      a.tcp = std::atoi(argv[++i]);
    } else if (key == "--ws" && i + 1 < argc) {
      a.ws = std::atoi(argv[++i]);
    } else if (key == "--threads" && i + 1 < argc) {
      a.threads = std::atoi(argv[++i]);
    } else if (key == "--addr" && i + 1 < argc) {
      a.addr = argv[++i];
```

```
    } else if (key == "--help") {
      std::cout << "Usage: ./peerlink_server --tcp 5555 --ws 8080 --threads 4
    --addr 0.0.0.0\n";
      std::exit(0);
    }
  }

  // Safety clamp: at least 1 thread.
  if (a.threads < 1) a.threads = 1;
  return a;
}

int main(int argc, char** argv) {
  // Parse CLI options.
  Args args = parse_args(argc, argv);

  // The central event loop for asynchronous I/O.
  boost::asio::io_context io;

  // Handle SIGINT/SIGTERM so Ctrl+C can stop gracefully.
  boost::asio::signal_set signals(io, SIGINT, SIGTERM);

  // When signal arrives, stop io_context which stops the servers.
  signals.async_wait([&](auto, auto) {
    util::Log::warn("shutdown_signal");
    io.stop();
  });

  // Create a shared room manager (lobby exists automatically).
  auto rooms = std::make_shared<core::RoomManager>();

  // Create servers.
  net::TcpServer tcp(io, rooms, args.addr,
    static_cast<std::uint16_t>(args.tcp));
  net::WsServer  ws (io, rooms, args.addr,
    static_cast<std::uint16_t>(args.ws));

  // Start listening.
  tcp.start();
  ws.start();

  util::Log::info("server_running threads=" + std::to_string(args.threads));

  // Run io_context on a thread pool.
  std::vector<std::thread> pool;
  pool.reserve(args.threads);
```

```
  for (int i = 0; i < args.threads; ++i) {
    pool.emplace_back([&]() {
      // Each thread runs the event loop.
      io.run();
    });
  }

  // Wait for all threads to finish.
  for (auto& t : pool) {
    t.join();
  }

  util::Log::info("server_stopped");
  return 0;
}
```

Chapter 6

Backend Code: include/core/CommandParser.hpp

6.1 Purpose

This header defines the command model (what clients send) and the parser interface. Both TCP and WebSocket sessions use the same parser so the protocol rules stay consistent.

6.2 Full Code with Line-by-Line Comments

```
// File: include/core/CommandParser.hpp
// Purpose: Define the command schema + parsing API for incoming JSON
   protocol.

#pragma once
// Why: Prevents multiple inclusion of this header in one compilation unit.

#include <string>
// Why: We use std::string for fields like type, nick, room, text.

#include <optional>
// Why: Parser returns std::optional<Command> to represent "parse failed".

#include <boost/json.hpp>
// Why: We parse and create JSON using Boost.JSON (boost::json).

namespace core {
// Why: Put core (non-network) logic in a separate namespace.

  // Command represents one request from a client to the server.
  struct Command {
```

```
  // 'type' decides what action this command represents.
  // Allowed: "ping", "name", "join", "message", "pm"
  std::string type;

  // For {"type":"name","nick":"ansh"}
  std::string nick;

  // For {"type":"join","room":"lobby"}
  std::string room;

  // For {"type":"message","text":"hello"}
  // Also used in pm: {"type":"pm","text":"..."}
  std::string text;

  // For {"type":"pm","to":"bob","text":"hi"}
  std::string to;
};

// ParseResult: either a valid Command or an error.
struct ParseResult {
  // ok==true means cmd contains valid fields for the command type.
  bool ok = false;

  // If ok==true, cmd is filled.
  Command cmd{};

  // If ok==false, error_code describes why parsing failed.
  // Example: "bad_json", "missing_type", "missing_field", "invalid_type"
  std::string error_code;
};

class CommandParser {
public:
  // Parse a JSON string into a ParseResult.
  // input: raw text containing JSON object (one line for TCP or WS frame
 for WebSocket)
  static ParseResult parse(const std::string& input);

private:
  // Helpers to read a string field from a JSON object.
  // Returns empty optional if missing or not a string.
  static std::optional<std::string> get_string(const boost::json::object&
 obj,
                                               const char* key);

  // Validate that the command has required fields for its type.
```

```
    // Returns error_code string if invalid; empty means ok.
    static std::string validate(const Command& c);
  };

} // namespace core
```

Chapter 7

Backend Code: src/core/CommandParser.cpp

7.1 Purpose

This file implements JSON parsing:

- converts raw JSON into a `Command`
- validates required fields depending on `type`
- returns a structured error code on failure

7.2 Full Code with Line-by-Line Comments

```
// File: src/core/CommandParser.cpp
// Purpose: Parse and validate incoming JSON protocol commands.

#include "core/CommandParser.hpp"
// Why: We implement the declarations from the header.

#include <cctype>
// Why: Sometimes we may want to check whitespace/emptiness robustly.

namespace core {
// Why: Keep definitions inside the same namespace as the declaration.

std::optional<std::string>
CommandParser::get_string(const boost::json::object& obj, const char* key) {
  // Look for the key in the JSON object.
  auto it = obj.find(key);
  // If key is not found, return empty.
  if (it == obj.end()) return std::nullopt;
```

```
  // Ensure the value is a JSON string.
  if (!it->value().is_string()) return std::nullopt;

  // Convert boost::json::string to std::string.
  return std::string(it->value().as_string().c_str());
}

std::string CommandParser::validate(const Command& c) {
  // Validate allowed command types first.
  if (c.type != "ping" &&
      c.type != "name" &&
      c.type != "join" &&
      c.type != "message" &&
      c.type != "pm") {
    // If type is unknown, return an error code.
    return "invalid_type";
  }

  // ping requires no extra fields.
  if (c.type == "ping") return "";

  // name requires nick.
  if (c.type == "name") {
    if (c.nick.empty()) return "missing_nick";
    return "";
  }

  // join requires room.
  if (c.type == "join") {
    if (c.room.empty()) return "missing_room";
    return "";
  }

  // message requires text.
  if (c.type == "message") {
    if (c.text.empty()) return "missing_text";
    return "";
  }

  // pm requires 'to' and 'text'.
  if (c.type == "pm") {
    if (c.to.empty()) return "missing_to";
    if (c.text.empty()) return "missing_text";
    return "";
  }
```

```
  // Should never reach here because type was validated above.
  return "invalid_type";
}

ParseResult CommandParser::parse(const std::string& input) {
  ParseResult out;
  // Default: out.ok=false, set ok=true only on success.

  // Quick check: empty input is invalid.
  if (input.empty()) {
    out.ok = false;
    out.error_code = "empty_input";
    return out;
  }

  // Parse JSON using Boost.JSON.
  boost::system::error_code ec;
  boost::json::value v = boost::json::parse(input, ec);

  // If parsing failed, return error.
  if (ec) {
    out.ok = false;
    out.error_code = "bad_json";
    return out;
  }

  // We expect a JSON object like {"type":"join",...}
  if (!v.is_object()) {
    out.ok = false;
    out.error_code = "not_object";
    return out;
  }

  // Access as object.
  const auto& obj = v.as_object();

  // Read type (required).
  auto type = get_string(obj, "type");
  if (!type) {
    out.ok = false;
    out.error_code = "missing_type";
    return out;
  }

  // Fill command fields; non-required fields can remain empty strings.
  Command c;
```

```
  c.type = *type;

  // Optional fields depending on command type:
  if (auto nick = get_string(obj, "nick")) c.nick = *nick;
  if (auto room = get_string(obj, "room")) c.room = *room;
  if (auto text = get_string(obj, "text")) c.text = *text;
  if (auto to   = get_string(obj, "to"))   c.to   = *to;

  // Validate fields for this command type.
  std::string err = validate(c);
  if (!err.empty()) {
    out.ok = false;
    out.error_code = err;
    return out;
  }

  // Success.
  out.ok = true;
  out.cmd = c;
  out.error_code.clear();
  return out;
}

} // namespace core
```

Chapter 8

Backend Code: include/core/ChatRoom.hpp

8.1 Purpose

`ChatRoom` is the core routing engine:

- tracks connected members (sessions) in a room
- broadcasts messages to all members
- supports PM by nickname lookup
- reports current user list

Networking is NOT inside this class: it only stores deliver callbacks.

8.2 Full Code with Line-by-Line Comments

```
// File: include/core/ChatRoom.hpp
// Purpose: Room membership + message fan-out (broadcast + private messaging).

#pragma once
// Why: Prevent multiple header inclusion.

#include <string>
// Why: Room name and nicknames are strings.

#include <vector>
// Why: We return user lists as std::vector<std::string>.

#include <unordered_map>
// Why: Map session id -> callback, and nickname -> session id.
```

```
#include <functional>
// Why: deliver callbacks are std::function<void(const std::string&)>.

#include <mutex>
// Why: Membership is shared across threads; protect with mutex.

#include <cstdint>
// Why: We use uint64_t for stable session ids.

namespace core {

  // DeliverFn: how the room sends a message to a session.
  // The session provides this callback and knows how to enqueue +
    async_write.
  using DeliverFn = std::function<void(const std::string&)>;

  class ChatRoom {
  public:
    // Construct room with a name.
    explicit ChatRoom(std::string name);

    // Get room name.
    const std::string& name() const;

    // Join: register a session id, nickname, and deliver callback.
    // Returns true on success.
    bool join(std::uint64_t session_id, const std::string& nick, DeliverFn
   deliver);

    // Leave: remove a session id from the room.
    void leave(std::uint64_t session_id);

    // Broadcast a message event to all sessions in the room.
    void broadcast(const std::string& json_event);

    // Private message: deliver to exactly one nickname.
    // Returns true if delivered, false if nickname not found.
    bool private_message(const std::string& to_nick, const std::string&
   json_event);

    // List current users (nicknames) in the room.
    std::vector<std::string> users() const;

    // Check if a nickname exists in this room.
    bool has_nick(const std::string& nick) const;
```

```
  private:
    // Room name.
    std::string name_;

    // Protect all member maps.
    mutable std::mutex mu_;

    // session_id -> deliver callback
    std::unordered_map<std::uint64_t, DeliverFn> deliver_by_id_;

    // session_id -> nickname
    std::unordered_map<std::uint64_t, std::string> nick_by_id_;

    // nickname -> session_id
    std::unordered_map<std::string, std::uint64_t> id_by_nick_;
  };

} // namespace core
```

Chapter 9

Backend Code: src/core/ChatRoom.cpp

9.1 Purpose

Implements membership + routing logic with thread-safety:

- joins/leaves update maps under a mutex
- broadcast uses a snapshot of callbacks (avoid holding lock during user code)
- PM finds target callback safely

9.2 Full Code with Line-by-Line Comments

```
// File: src/core/ChatRoom.cpp
// Purpose: Implementation of ChatRoom membership and message fan-out.

#include "core/ChatRoom.hpp"
// Why: Implements the header declarations.

namespace core {

ChatRoom::ChatRoom(std::string name)
  : name_(std::move(name)) {
  // Why move: avoid copying the room name string.
}

const std::string& ChatRoom::name() const {
  // Return reference to internal name string.
  return name_;
}
```

```
bool ChatRoom::join(std::uint64_t session_id, const std::string& nick,
    DeliverFn deliver) {
  // Lock to protect membership structures.
  std::lock_guard<std::mutex> lock(mu_);

  // Enforce non-empty nickname.
  if (nick.empty()) return false;

  // If this nickname is already used, reject to prevent ambiguity.
  if (id_by_nick_.find(nick) != id_by_nick_.end()) return false;

  // Insert mappings.
  deliver_by_id_[session_id] = std::move(deliver);
  nick_by_id_[session_id] = nick;
  id_by_nick_[nick] = session_id;

  return true;
}

void ChatRoom::leave(std::uint64_t session_id) {
  std::lock_guard<std::mutex> lock(mu_);

  // Remove deliver callback.
  deliver_by_id_.erase(session_id);

  // Remove nickname mappings if present.
  auto it = nick_by_id_.find(session_id);
  if (it != nick_by_id_.end()) {
    // erase reverse mapping nick->id
    id_by_nick_.erase(it->second);
    // erase id->nick
    nick_by_id_.erase(it);
  }
}

void ChatRoom::broadcast(const std::string& json_event) {
  // We do NOT want to hold the mutex while calling callbacks,
  // because callbacks may do network operations or enqueue work.
  // So we snapshot the callbacks under lock, then call outside.
  std::vector<DeliverFn> targets;

  {
    std::lock_guard<std::mutex> lock(mu_);
    targets.reserve(deliver_by_id_.size());
    for (auto& kv : deliver_by_id_) {
      targets.push_back(kv.second);
```

```
    }
  }

  // Now call all deliver callbacks without holding the lock.
  for (auto& deliver : targets) {
    deliver(json_event);
  }
}

bool ChatRoom::private_message(const std::string& to_nick, const std::string&
    json_event) {
  DeliverFn target;

  {
    std::lock_guard<std::mutex> lock(mu_);

    // Find recipient by nickname.
    auto it = id_by_nick_.find(to_nick);
    if (it == id_by_nick_.end()) return false;

    // Find deliver callback for that session id.
    auto dit = deliver_by_id_.find(it->second);
    if (dit == deliver_by_id_.end()) return false;

    // Copy callback out to call outside lock.
    target = dit->second;
  }

  // Deliver outside lock.
  target(json_event);
  return true;
}

std::vector<std::string> ChatRoom::users() const {
  std::vector<std::string> out;

  std::lock_guard<std::mutex> lock(mu_);
  out.reserve(nick_by_id_.size());

  for (auto const& kv : nick_by_id_) {
    out.push_back(kv.second);
  }
  return out;
}

bool ChatRoom::has_nick(const std::string& nick) const {
```

```
  std::lock_guard<std::mutex> lock(mu_);
  return id_by_nick_.find(nick) != id_by_nick_.end();
}

} // namespace core
```

Chapter 10

Backend Code: include/core/RoomManager.hpp

10.1 Purpose

`RoomManager` is a registry of rooms:

- guarantees `lobby` always exists
- creates custom rooms on-demand
- returns shared pointers so rooms can be safely shared by many sessions

10.2 Full Code with Line-by-Line Comments

```
// File: include/core/RoomManager.hpp
// Purpose: Central registry for chat rooms (lobby + custom rooms).

#pragma once
// Why: Prevent multiple inclusion.

#include <memory>
// Why: Rooms are shared across sessions using std::shared_ptr.

#include <string>
// Why: Room names are strings.

#include <unordered_map>
// Why: Map room name -> ChatRoom.

#include <mutex>
// Why: Registry is shared; protect with mutex.

#include "core/ChatRoom.hpp"
```

```
// Why: RoomManager creates and returns ChatRoom objects.

namespace core {

  class RoomManager {
  public:
    // Construct and ensure lobby exists.
    RoomManager();

    // Get the lobby room (always exists).
    std::shared_ptr<ChatRoom> lobby();

    // Find or create a room by name.
    // If name is empty, returns lobby.
    std::shared_ptr<ChatRoom> get_or_create(const std::string& name);

    // Optional: list room names (useful for debugging/metrics).
    // Not required by protocol, but helpful in reports/testing.
    std::vector<std::string> room_names() const;

  private:
    // Mutex protecting rooms_.
    mutable std::mutex mu_;

    // Registry: room name -> room object.
    std::unordered_map<std::string, std::shared_ptr<ChatRoom>> rooms_;

    // Cached lobby pointer to avoid repeated lookup.
    std::shared_ptr<ChatRoom> lobby_;
  };

} // namespace core
```

Chapter 11

Backend Code: src/core/RoomManager.cpp

11.1 Purpose

Implements safe room creation and lookup:

- creates `lobby` exactly once
- ensures multiple threads cannot create duplicate rooms
- returns shared pointers so sessions can hold rooms safely

11.2 Full Code with Line-by-Line Comments

```
// File: src/core/RoomManager.cpp
// Purpose: Implementation of room registry with thread safety.

#include "core/RoomManager.hpp"
// Why: Implements the header declarations.

#include <algorithm>
// Why: For any optional operations like sorting room names.

namespace core {

RoomManager::RoomManager() {
  // Create lobby on construction so it always exists.
  std::lock_guard<std::mutex> lock(mu_);

  // Allocate a ChatRoom object named "lobby".
  lobby_ = std::make_shared<ChatRoom>("lobby");

  // Store it in the registry map.
```

```
  rooms_["lobby"] = lobby_;
}

std::shared_ptr<ChatRoom> RoomManager::lobby() {
  // Lobby pointer is stable; still safe to return without extra work.
  // We lock to be conservative (thread-safe if future changes happen).
  std::lock_guard<std::mutex> lock(mu_);
  return lobby_;
}

std::shared_ptr<ChatRoom> RoomManager::get_or_create(const std::string& name)
    {
  // If name is empty, fallback to lobby by protocol convention.
  if (name.empty()) return lobby();

  std::lock_guard<std::mutex> lock(mu_);

  // Check if room exists already.
  auto it = rooms_.find(name);
  if (it != rooms_.end()) {
    // Return existing room.
    return it->second;
  }

  // Create a new room and insert into registry.
  auto room = std::make_shared<ChatRoom>(name);
  rooms_[name] = room;
  return room;
}

std::vector<std::string> RoomManager::room_names() const {
  std::vector<std::string> names;

  std::lock_guard<std::mutex> lock(mu_);
  names.reserve(rooms_.size());

  for (auto const& kv : rooms_) {
    names.push_back(kv.first);
  }

  // Sorting makes output stable across runs (useful for tests/logging).
  std::sort(names.begin(), names.end());
  return names;
}

} // namespace core
```

Chapter 12

Backend Code: include/net/TcpServer.hpp

12.1 Purpose

`TcpServer` listens on a TCP port and accepts new connections. For each accepted socket it creates a `TcpSession`.

This file contains only acceptor logic; per-client behavior is in `TcpSession`.

12.2 Full Code with Line-by-Line Comments

```
// File: include/net/TcpServer.hpp
// Purpose: Async TCP acceptor that spawns TcpSession objects.

#pragma once
// Why: Prevent multiple inclusion.

#include <boost/asio.hpp>
// Why: Provides tcp::acceptor, io_context, endpoints, async_accept.

#include <memory>
// Why: We hold shared_ptr to RoomManager to share across sessions.

#include <cstdint>
// Why: Use uint16_t for port.

#include "core/RoomManager.hpp"
// Why: Tcp sessions need access to rooms.

namespace net {

  class TcpServer {
```

```
  public:
    // Construct TCP server.
    // io: Asio event loop used for async ops.
    // rooms: shared room registry.
    // addr: bind address (e.g., 0.0.0.0)
    // port: listening port
    TcpServer(boost::asio::io_context& io,
              std::shared_ptr<core::RoomManager> rooms,
              const std::string& addr,
              std::uint16_t port);

    // Start accepting connections.
    void start();

  private:
    // Continue accepting connections (internal loop).
    void do_accept();

    // io_context reference for async operations.
    boost::asio::io_context& io_;

    // TCP acceptor that listens on a port.
    boost::asio::ip::tcp::acceptor acceptor_;

    // Shared room manager (lobby + custom rooms).
    std::shared_ptr<core::RoomManager> rooms_;
  };

} // namespace net
```

Chapter 13

Backend Code: src/net/TcpServer.cpp

13.1 Purpose

Implements:

- bind/listen logic
- async accept loop
- creation of `TcpSession` objects
- logging for observability

13.2 Full Code with Line-by-Line Comments

```
// File: src/net/TcpServer.cpp
// Purpose: Implementation of async TCP accept loop.

#include "net/TcpServer.hpp"
// Why: Implements TcpServer declarations.

#include "net/TcpSession.hpp"
// Why: TcpServer spawns TcpSession on each accepted socket.

#include "util/Log.hpp"
// Why: Log JSON events like listen + accept failures.

namespace net {

TcpServer::TcpServer(boost::asio::io_context& io,
                     std::shared_ptr<core::RoomManager> rooms,
                     const std::string& addr,
```

```
                    std::uint16_t port)
  : io_(io),
    // Create acceptor bound to io_context.
    acceptor_(io),
    rooms_(std::move(rooms)) {

  // Parse the bind address (e.g., "0.0.0.0").
  boost::system::error_code ec;
  auto address = boost::asio::ip::make_address(addr, ec);
  if (ec) {
    // If address parsing fails, throw because server cannot start.
    throw std::runtime_error("Invalid bind address: " + addr);
  }

  // Create endpoint (IP + port).
  boost::asio::ip::tcp::endpoint ep(address, port);

  // Open acceptor with protocol (v4 or v6).
  acceptor_.open(ep.protocol(), ec);
  if (ec) throw std::runtime_error("acceptor.open failed");

  // Allow quick restart (reuse address).
  acceptor_.set_option(boost::asio::socket_base::reuse_address(true), ec);
  if (ec) throw std::runtime_error("acceptor.set_option failed");

  // Bind to the endpoint.
  acceptor_.bind(ep, ec);
  if (ec) throw std::runtime_error("acceptor.bind failed");

  // Start listening for connections.
  acceptor_.listen(boost::asio::socket_base::max_listen_connections, ec);
  if (ec) throw std::runtime_error("acceptor.listen failed");
}

void TcpServer::start() {
  // Log that TCP server is listening.
  util::Log::info("tcp_listen addr=" +
                  acceptor_.local_endpoint().address().to_string() +
                  " port=" +
    std::to_string(acceptor_.local_endpoint().port()));

  // Begin accept loop.
  do_accept();
}

void TcpServer::do_accept() {
```

```
  // Create a socket for the next client.
  auto sock = std::make_shared<boost::asio::ip::tcp::socket>(io_);

  // Start async accept.
  acceptor_.async_accept(*sock, [this, sock](boost::system::error_code ec) {
    if (ec) {
      // Log accept failure and continue accepting.
      util::Log::warn("tcp_accept_failed code=" + std::to_string(ec.value()));
      do_accept();
      return;
    }

    // On success: create a new TcpSession and start it.
    // TcpSession takes ownership of the socket.
    std::make_shared<net::TcpSession>(std::move(*sock), rooms_)->start();

    // Continue accepting next connections.
    do_accept();
  });
}

} // namespace net
```

Chapter 14

Backend Code: include/net/TcpSession.hpp

14.1 Purpose

`TcpSession` represents exactly one connected TCP client. It is responsible for:

- async reading newline-delimited JSON
- parsing JSON commands using `core::CommandParser`
- enforcing protocol order (name → join → message)
- joining/leaving a room (via `RoomManager`/`ChatRoom`)
- sending outgoing JSON using a write queue (non-blocking)

14.2 Full Code with Line-by-Line Comments

```
// File: include/net/TcpSession.hpp
// Purpose: One TCP client connection with async read/write loops.

#pragma once
// Why: Prevent multiple inclusion.

#include <boost/asio.hpp>
// Why: Provides tcp::socket, async_read_until, async_write, streambuf.

#include <deque>
// Why: Outgoing message queue.

#include <memory>
// Why: Enable shared_from_this for lifetime safety.
```

```
#include <string>
// Why: nick, room, and JSON strings.

#include <cstdint>
// Why: session id type.

#include "core/RoomManager.hpp"
// Why: sessions use RoomManager to get rooms.

#include "core/ChatRoom.hpp"
// Why: sessions hold a pointer to current room.

#include "core/CommandParser.hpp"
// Why: parse incoming JSON protocol commands.

namespace net {

  class TcpSession : public std::enable_shared_from_this<TcpSession> {
  public:
    // Construct with an already-accepted TCP socket.
    TcpSession(boost::asio::ip::tcp::socket socket,
               std::shared_ptr<core::RoomManager> rooms);

    // Start the session: join default room and begin async reads.
    void start();

    // Deliver a JSON event to this client (called by ChatRoom).
    void deliver(const std::string& json_event);

  private:
    // Begin async read loop for newline-delimited messages.
    void do_read_line();

    // Handle one complete input line.
    void on_line(const std::string& line);

    // Begin async write loop if not already writing.
    void do_write();

    // Helper: send an "error" event JSON.
    void send_error(const std::string& code);

    // Helper: send an "ok" event JSON.
    void send_ok(const std::string& what);

    // Helper: build a "joined" event JSON.
```

```
    std::string mk_joined_event(const std::string& room_name,
                                const std::vector<std::string>& users);

  private:
    // Underlying socket for this client.
    boost::asio::ip::tcp::socket socket_;

    // Asio buffer used by async_read_until.
    boost::asio::streambuf inbuf_;

    // Outgoing queue of JSON strings (each ends with '\n' for TCP).
    std::deque<std::string> outq_;

    // Flag: is an async write currently in-flight?
    bool writing_ = false;

    // Shared room manager.
    std::shared_ptr<core::RoomManager> rooms_;

    // Current room pointer (null until joined).
    std::shared_ptr<core::ChatRoom> room_;

    // Session id unique number.
    std::uint64_t id_ = 0;

    // Nickname for this session (must be set before join/message).
    std::string nick_;

    // Whether nickname is set.
    bool has_nick_ = false;

    // Whether joined a room.
    bool joined_ = false;

    // Static counter used to allocate unique ids.
    static std::atomic<std::uint64_t> next_id_;
  };

} // namespace net
```

Chapter 15

Backend Code: src/net/TcpSession.cpp

15.1 Purpose

Implements TCP protocol execution:

- reads JSON lines using `async_read_until`
- parses commands
- routes to ChatRoom methods
- uses write queue so no blocking occurs

15.2 Full Code with Line-by-Line Comments

```
// File: src/net/TcpSession.cpp
// Purpose: Implementation of the per-connection TCP session.

#include "net/TcpSession.hpp"
// Why: Implements the session declarations.

#include "util/Log.hpp"
// Why: JSON logging for observability.

#include <boost/json.hpp>
// Why: Build JSON responses/events.

#include <atomic>
// Why: session id counter.

namespace net {
```

```
// Define static counter for ids.
std::atomic<std::uint64_t> TcpSession::next_id_{1};

TcpSession::TcpSession(boost::asio::ip::tcp::socket socket,
                       std::shared_ptr<core::RoomManager> rooms)
  : socket_(std::move(socket)),
    rooms_(std::move(rooms)) {
  // Assign a unique session id.
  id_ = next_id_.fetch_add(1);
}

void TcpSession::start() {
  // Log connection established.
  util::Log::info("tcp_connected id=" + std::to_string(id_));

  // Start reading line-delimited JSON.
  do_read_line();
}

void TcpSession::do_read_line() {
  // Keep this object alive during async op.
  auto self = shared_from_this();

  // Read until newline delimiter.
  boost::asio::async_read_until(socket_, inbuf_, '\n',
    [self](boost::system::error_code ec, std::size_t n) {
      if (ec) {
        // Client disconnected or read error.
        util::Log::info("tcp_disconnected id=" + std::to_string(self->id_));
        // Leave room if joined.
        if (self->room_) self->room_->leave(self->id_);
        return;
      }

      // Extract exactly one line from streambuf.
      std::istream is(&self->inbuf_);
      std::string line;
      std::getline(is, line); // removes '\n'

      util::Log::info("tcp_rx id=" + std::to_string(self->id_) +
                      " bytes=" + std::to_string(line.size()));

      // Handle the line as one JSON command.
      self->on_line(line);

      // Continue reading next line.
```

```
      self->do_read_line();
    }
  );
}

std::string TcpSession::mk_joined_event(const std::string& room_name,
                                        const std::vector<std::string>& users)
    {
  // Build {"type":"joined","data":{"room":..., "users":[...]}}
  boost::json::object data;
  data["room"] = room_name;

  boost::json::array arr;
  for (auto const& u : users) {
    // Boost.JSON needs boost::json::value, not std::string directly.
    arr.push_back(boost::json::value(u));
  }
  data["users"] = std::move(arr);

  boost::json::object ev;
  ev["type"] = "joined";
  ev["data"] = std::move(data);

  // For TCP we append newline so client can read line by line.
  return boost::json::serialize(ev) + "\n";
}

void TcpSession::send_error(const std::string& code) {
  boost::json::object data;
  data["code"] = code;

  boost::json::object ev;
  ev["type"] = "error";
  ev["data"] = std::move(data);

  deliver(boost::json::serialize(ev) + "\n");
}

void TcpSession::send_ok(const std::string& what) {
  boost::json::object data;
  data["what"] = what;

  boost::json::object ev;
  ev["type"] = "ok";
  ev["data"] = std::move(data);
```

```
  deliver(boost::json::serialize(ev) + "\n");
}

void TcpSession::on_line(const std::string& line) {
  // Parse the incoming JSON.
  auto res = core::CommandParser::parse(line);

  if (!res.ok) {
    // Parser failed: respond with error event.
    send_error(res.error_code);
    return;
  }

  const auto& cmd = res.cmd;

  // Handle ping.
  if (cmd.type == "ping") {
    send_ok("pong");
    return;
  }

  // Handle name set.
  if (cmd.type == "name") {
    nick_ = cmd.nick;
    has_nick_ = true;

    util::Log::info("tcp_name id=" + std::to_string(id_) + " nick=" + nick_);
    send_ok("name_set");
    return;
  }

  // For any command after this point, nickname must be set.
  if (!has_nick_) {
    send_error("nick_required");
    return;
  }

  // Handle join.
  if (cmd.type == "join") {
    // If already joined, leave current room first.
    if (room_) {
      room_->leave(id_);
      room_.reset();
      joined_ = false;
    }
```

```
  // Get or create requested room, default is lobby if empty.
  room_ = rooms_->get_or_create(cmd.room);

  // Attempt to join with nickname uniqueness enforced by ChatRoom.
  bool ok = room_->join(id_, nick_,
    [self = shared_from_this()](const std::string& ev) {
      // Deliver callback called by ChatRoom.
      self->deliver(ev + "\n"); // ensure newline framing
    }
  );

  if (!ok) {
    send_error("nick_in_use_or_invalid");
    room_.reset();
    return;
  }

  joined_ = true;
  util::Log::info("tcp_join id=" + std::to_string(id_) + " room=" +
 room_->name());

  // Send joined event with current user list.
  deliver(mk_joined_event(room_->name(), room_->users()));
  return;
}

// For message/pm, must have joined a room.
if (!joined_ || !room_) {
  send_error("join_required");
  return;
}

// Handle pm.
if (cmd.type == "pm") {
  // Build event {"type":"message","data":{...,"pm":true}}
  boost::json::object data;
  data["room"] = room_->name();
  data["from"] = nick_;
  data["to"] = cmd.to;
  data["text"] = cmd.text;
  data["pm"] = true;

  boost::json::object ev;
  ev["type"] = "message";
  ev["data"] = std::move(data);
```

```
    std::string payload = boost::json::serialize(ev);

    bool ok = room_->private_message(cmd.to, payload);
    util::Log::info("tcp_pm id=" + std::to_string(id_) + " to=" + cmd.to +
                    " ok=" + std::string(ok ? "true" : "false"));

    if (!ok) send_error("pm_target_not_found");
    return;
  }

  // Handle broadcast message.
  if (cmd.type == "message") {
    boost::json::object data;
    data["room"] = room_->name();
    data["from"] = nick_;
    data["text"] = cmd.text;
    data["pm"] = false;

    boost::json::object ev;
    ev["type"] = "message";
    ev["data"] = std::move(data);

    std::string payload = boost::json::serialize(ev);

    util::Log::info("tcp_msg id=" + std::to_string(id_) + " room=" +
   room_->name());

    // Broadcast to everyone in the room.
    room_->broadcast(payload);
    return;
  }

  // If we reach here: unknown validated type (should not happen).
  send_error("invalid_type");
}

void TcpSession::deliver(const std::string& json_event) {
  // Enqueue outgoing data and start write loop if idle.
  bool idle = outq_.empty();
  outq_.push_back(json_event);

  if (idle && !writing_) {
    do_write();
  }
}
```

```
void TcpSession::do_write() {
  // If nothing to write, stop.
  if (outq_.empty()) {
    writing_ = false;
    return;
  }

  writing_ = true;

  auto self = shared_from_this();
  auto& msg = outq_.front();

  // Async write one message.
  boost::asio::async_write(socket_, boost::asio::buffer(msg),
    [self](boost::system::error_code ec, std::size_t) {
      if (ec) {
        util::Log::warn("tcp_write_error id=" + std::to_string(self->id_) +
                        " code=" + std::to_string(ec.value()));
        // On write error, disconnect and cleanup.
        if (self->room_) self->room_->leave(self->id_);
        return;
      }

      // Remove sent message.
      self->outq_.pop_front();

      // Write next if any.
      self->do_write();
    }
  );
}

} // namespace net
```

Chapter 16

Backend Code: include/net/WsServer.hpp

16.1 Purpose

`WsServer` listens on a TCP port for WebSocket upgrade requests. It accepts raw TCP sockets and creates `WsSession` objects that do the handshake.

16.2 Full Code with Line-by-Line Comments

```
// File: include/net/WsServer.hpp
// Purpose: Async acceptor for WebSocket clients (Boost.Beast sessions).

#pragma once

#include <boost/asio.hpp>
// Why: TCP acceptor and io_context.

#include <memory>
// Why: Share RoomManager across sessions.

#include <cstdint>
// Why: port type.

#include "core/RoomManager.hpp"
// Why: sessions use rooms.

namespace net {

  class WsServer {
  public:
    WsServer(boost::asio::io_context& io,
             std::shared_ptr<core::RoomManager> rooms,
```

```
            const std::string& addr,
            std::uint16_t port);

    void start();

  private:
    void do_accept();

    boost::asio::io_context& io_;
    boost::asio::ip::tcp::acceptor acceptor_;
    std::shared_ptr<core::RoomManager> rooms_;
  };

} // namespace net
```

Chapter 17

Backend Code: src/net/WsServer.cpp

17.1 Purpose

Implements:

- bind/listen on WS port
- async accept loop
- spawns `WsSession` per connection

17.2 Full Code with Line-by-Line Comments

```
// File: src/net/WsServer.cpp
// Purpose: Implementation of WebSocket accept loop.

#include "net/WsServer.hpp"
// Why: Implements WsServer.

#include "net/WsSession.hpp"
// Why: WsServer spawns WsSession on accepted socket.

#include "util/Log.hpp"
// Why: JSON logging.

namespace net {

WsServer::WsServer(boost::asio::io_context& io,
                   std::shared_ptr<core::RoomManager> rooms,
                   const std::string& addr,
                   std::uint16_t port)
  : io_(io),
```

```
    acceptor_(io),
    rooms_(std::move(rooms)) {

  boost::system::error_code ec;
  auto address = boost::asio::ip::make_address(addr, ec);
  if (ec) throw std::runtime_error("Invalid bind address: " + addr);

  boost::asio::ip::tcp::endpoint ep(address, port);

  acceptor_.open(ep.protocol(), ec);
  if (ec) throw std::runtime_error("ws acceptor.open failed");

  acceptor_.set_option(boost::asio::socket_base::reuse_address(true), ec);
  if (ec) throw std::runtime_error("ws acceptor.set_option failed");

  acceptor_.bind(ep, ec);
  if (ec) throw std::runtime_error("ws acceptor.bind failed");

  acceptor_.listen(boost::asio::socket_base::max_listen_connections, ec);
  if (ec) throw std::runtime_error("ws acceptor.listen failed");
}

void WsServer::start() {
  util::Log::info("ws_listen addr=" +
                  acceptor_.local_endpoint().address().to_string() +
                  " port=" +
    std::to_string(acceptor_.local_endpoint().port()));
  do_accept();
}

void WsServer::do_accept() {
  auto sock = std::make_shared<boost::asio::ip::tcp::socket>(io_);

  acceptor_.async_accept(*sock, [this, sock](boost::system::error_code ec) {
    if (ec) {
      util::Log::warn("ws_accept_failed code=" + std::to_string(ec.value()));
      do_accept();
      return;
    }

    // Spawn WebSocket session which will do async_accept handshake.
    std::make_shared<net::WsSession>(std::move(*sock), rooms_)->start();

    // Continue accept loop.
    do_accept();
  });
```

```
}

} // namespace net
```

Chapter 18

Backend Code: include/net/WsSession.hpp

18.1 Purpose

`WsSession` represents exactly one connected WebSocket client. It is responsible for:

- performing WebSocket handshake (HTTP upgrade) via Boost.Beast
- reading text frames asynchronously
- parsing incoming JSON commands using `core::CommandParser`
- enforcing protocol order (name → join → message)
- routing broadcast/PM through `ChatRoom`
- using a write queue so message fan-out never blocks

18.2 Full Code with Line-by-Line Comments

```
// File: include/net/WsSession.hpp
// Purpose: One WebSocket client connection using Boost.Beast.

#pragma once
// Why: Prevent multiple inclusion.

#include <boost/asio.hpp>
// Why: Provides io_context executor, strand, tcp::socket.

#include <boost/beast.hpp>
// Why: Provides websocket::stream and buffers.

#include <deque>
```

```
// Why: Outgoing write queue.

#include <memory>
// Why: shared_from_this for lifetime safety.

#include <string>
// Why: JSON strings, nickname, room.

#include <cstdint>
// Why: session id type.

#include <atomic>
// Why: global session id generator.

#include "core/RoomManager.hpp"
// Why: sessions need access to room registry.

#include "core/ChatRoom.hpp"
// Why: sessions join a ChatRoom and use broadcast/PM.

#include "core/CommandParser.hpp"
// Why: parse incoming JSON protocol commands.

namespace net {

  class WsSession : public std::enable_shared_from_this<WsSession> {
  public:
    // Construct from an accepted TCP socket.
    // WsSession will wrap it into websocket::stream internally.
    WsSession(boost::asio::ip::tcp::socket socket,
              std::shared_ptr<core::RoomManager> rooms);

    // Start: perform WebSocket accept handshake, then begin read loop.
    void start();

    // Deliver a JSON event to this session (called by ChatRoom).
    void deliver(const std::string& json_event);

  private:
    // After handshake, begin async read loop.
    void do_read();

    // Handle text frame received from client.
    void on_text(const std::string& text);

    // Begin draining write queue (async_write).
```

```
  void do_write();

  // Helpers to send ok/error.
  void send_error(const std::string& code);
  void send_ok(const std::string& what);

  // Helper: build joined event JSON string.
  std::string mk_joined_event(const std::string& room_name,
                              const std::vector<std::string>& users);

private:
  // WebSocket stream (wraps tcp socket).
  boost::beast::websocket::stream<boost::beast::tcp_stream> ws_;

  // Strand ensures handlers for this session do not run concurrently.
  // This is important because outq_ and state are not protected by a mutex.
  boost::asio::strand<boost::asio::any_io_executor> strand_;

  // Buffer for incoming frames.
  boost::beast::flat_buffer inbuf_;

  // Outgoing queue of JSON strings (each is one WS text frame payload).
  std::deque<std::string> outq_;

  // Flag: write in-flight?
  bool writing_ = false;

  // Shared room manager.
  std::shared_ptr<core::RoomManager> rooms_;

  // Current room.
  std::shared_ptr<core::ChatRoom> room_;

  // Session id.
  std::uint64_t id_ = 0;

  // Nickname.
  std::string nick_;
  bool has_nick_ = false;

  // Join state.
  bool joined_ = false;

  // Static id counter.
  static std::atomic<std::uint64_t> next_id_;
};
```

```
} // namespace net
```

Chapter 19

Backend Code: src/net/WsSession.cpp

19.1 Purpose

Implements the WebSocket session:

- `async_accept` handshake
- `async_read` for incoming text frames
- parsing commands and routing to room
- non-blocking delivery with a per-session write queue
- strand-based concurrency safety

19.2 Full Code with Line-by-Line Comments

```
// File: src/net/WsSession.cpp
// Purpose: Implementation of WebSocket session behavior.

#include "net/WsSession.hpp"
// Why: Implements WsSession.

#include "util/Log.hpp"
// Why: JSON logging for observability.

#include <boost/json.hpp>
// Why: Build JSON responses/events.

namespace net {

// Define static id counter.
```

```
std::atomic<std::uint64_t> WsSession::next_id_{1};

WsSession::WsSession(boost::asio::ip::tcp::socket socket,
                     std::shared_ptr<core::RoomManager> rooms)
  : ws_(std::move(socket)),                        // wrap tcp socket into
   Beast stream
    strand_(ws_.get_executor()),                    // strand uses same
   executor as ws_
    rooms_(std::move(rooms)) {

  // Assign unique id.
  id_ = next_id_.fetch_add(1);

  // Optional websocket settings:
  ws_.text(true); // we send/receive text frames (JSON strings).
}

void WsSession::start() {
  // Keep object alive.
  auto self = shared_from_this();

  // Perform WebSocket handshake asynchronously.
  ws_.async_accept(
    boost::asio::bind_executor(strand_,
      [self](boost::system::error_code ec) {
        if (ec) {
          util::Log::error("ws_accept_failed id=" + std::to_string(self->id_)
   +
                           " code=" + std::to_string(ec.value()));
          return;
        }

        util::Log::info("ws_connected id=" + std::to_string(self->id_));

        // Start read loop after successful handshake.
        self->do_read();
      }
    )
  );
}

void WsSession::do_read() {
  auto self = shared_from_this();

  // Read next frame.
  ws_.async_read(inbuf_,
```

```
    boost::asio::bind_executor(strand_,
      [self](boost::system::error_code ec, std::size_t bytes) {
        if (ec) {
          // Connection closed or error.
          util::Log::info("ws_disconnected id=" + std::to_string(self->id_) +
                          " code=" + std::to_string(ec.value()));

          // Cleanup room membership.
          if (self->room_) self->room_->leave(self->id_);
          return;
        }

        // Convert buffer to string.
        std::string text =
    boost::beast::buffers_to_string(self->inbuf_.data());
        self->inbuf_.consume(self->inbuf_.size());

        util::Log::info("ws_rx id=" + std::to_string(self->id_) +
                        " bytes=" + std::to_string(bytes));

        // Handle the JSON command.
        self->on_text(text);

        // Continue reading.
        self->do_read();
      }
    )
  );
}

std::string WsSession::mk_joined_event(const std::string& room_name,
                                       const std::vector<std::string>& users) {
  // Build {"type":"joined","data":{"room":..., "users":[...]}}
  boost::json::object data;
  data["room"] = room_name;

  boost::json::array arr;
  for (auto const& u : users) {
    // Need boost::json::value wrapper.
    arr.push_back(boost::json::value(u));
  }
  data["users"] = std::move(arr);

  boost::json::object ev;
  ev["type"] = "joined";
  ev["data"] = std::move(data);
```

```
  return boost::json::serialize(ev);
}

void WsSession::send_error(const std::string& code) {
  boost::json::object data;
  data["code"] = code;

  boost::json::object ev;
  ev["type"] = "error";
  ev["data"] = std::move(data);

  deliver(boost::json::serialize(ev));
}

void WsSession::send_ok(const std::string& what) {
  boost::json::object data;
  data["what"] = what;

  boost::json::object ev;
  ev["type"] = "ok";
  ev["data"] = std::move(data);

  deliver(boost::json::serialize(ev));
}

void WsSession::on_text(const std::string& text) {
  // Parse JSON command.
  auto res = core::CommandParser::parse(text);

  if (!res.ok) {
    send_error(res.error_code);
    return;
  }

  const auto& cmd = res.cmd;

  // ping -> ok pong
  if (cmd.type == "ping") {
    util::Log::info("ws_ping_pong id=" + std::to_string(id_));
    send_ok("pong");
    return;
  }

  // name -> set nickname
  if (cmd.type == "name") {
```

```
    nick_ = cmd.nick;
    has_nick_ = true;

    util::Log::info("ws_name id=" + std::to_string(id_) + " nick=" + nick_);
    send_ok("name_set");
    return;
  }

  // Require nick for other commands.
  if (!has_nick_) {
    send_error("nick_required");
    return;
  }

  // join command
  if (cmd.type == "join") {
    // Leave existing room if joined.
    if (room_) {
      room_->leave(id_);
      room_.reset();
      joined_ = false;
    }

    // Get requested room (default lobby if empty).
    room_ = rooms_->get_or_create(cmd.room);

    // Attempt to join room with unique nickname.
    bool ok = room_->join(id_, nick_,
      [self = shared_from_this()](const std::string& ev) {
        // Deliver callback from room -> enqueue into this session.
        self->deliver(ev);
      }
    );

    if (!ok) {
      send_error("nick_in_use_or_invalid");
      room_.reset();
      return;
    }

    joined_ = true;
    util::Log::info("ws_join id=" + std::to_string(id_) + " room=" +
   room_->name());

    // Reply with joined event and user list snapshot.
    deliver(mk_joined_event(room_->name(), room_->users()));
```

```
  return;
}

// Must be joined before message/pm.
if (!joined_ || !room_) {
  send_error("join_required");
  return;
}

// PM routing.
if (cmd.type == "pm") {
  boost::json::object data;
  data["room"] = room_->name();
  data["from"] = nick_;
  data["to"]   = cmd.to;
  data["text"] = cmd.text;
  data["pm"]   = true;

  boost::json::object ev;
  ev["type"] = "message";
  ev["data"] = std::move(data);

  std::string payload = boost::json::serialize(ev);

  bool ok = room_->private_message(cmd.to, payload);
  util::Log::info("ws_pm id=" + std::to_string(id_) +
                  " to=" + cmd.to +
                  " ok=" + std::string(ok ? "true" : "false"));

  if (!ok) send_error("pm_target_not_found");
  return;
}

// Broadcast message.
if (cmd.type == "message") {
  boost::json::object data;
  data["room"] = room_->name();
  data["from"] = nick_;
  data["text"] = cmd.text;
  data["pm"]   = false;

  boost::json::object ev;
  ev["type"] = "message";
  ev["data"] = std::move(data);

  std::string payload = boost::json::serialize(ev);
```

```
    util::Log::info("ws_msg id=" + std::to_string(id_) +
                    " room=" + room_->name());

    room_->broadcast(payload);
    return;
  }

  // Should not reach.
  send_error("invalid_type");
}

void WsSession::deliver(const std::string& json_event) {
  // All state changes must run in the session strand.
  auto self = shared_from_this();
  boost::asio::dispatch(strand_, [self, json_event] {
    bool idle = self->outq_.empty();
    self->outq_.push_back(json_event);

    if (idle && !self->writing_) {
      self->do_write();
    }
  });
}

void WsSession::do_write() {
  if (outq_.empty()) {
    writing_ = false;
    return;
  }

  writing_ = true;

  auto self = shared_from_this();
  auto msg = outq_.front(); // copy to keep buffer stable during async_write

  ws_.async_write(boost::asio::buffer(msg),
    boost::asio::bind_executor(strand_,
      [self, msg](boost::system::error_code ec, std::size_t) {
        if (ec) {
          util::Log::warn("ws_write_error id=" + std::to_string(self->id_) +
                          " code=" + std::to_string(ec.value()));
          if (self->room_) self->room_->leave(self->id_);
          return;
        }
```

```
        // Pop front (the message we sent).
        self->outq_.pop_front();

        // Continue draining.
        self->do_write();
      }
    )
  );
}

} // namespace net
```

Chapter 20

Frontend Code: web-react/index.html

20.1 Purpose

This is the HTML entry point used by Vite. It loads the React app into the `#root` element.

20.2 Full Code with Line-by-Line Comments

```
<!-- File: web-react/index.html -->
<!-- Purpose: Main HTML shell for the Peerlink React frontend. -->

<!doctype html>
<!-- Why: HTML5 document type. -->

<html lang="en">
<!-- Why: Language hint for browsers, accessibility, and SEO. -->

  <head>
    <!-- Why: Metadata and page settings. -->
    <meta charset="UTF-8" />
    <!-- Why: UTF-8 character encoding. -->

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- Why: Responsive layout on mobile devices. -->

    <title>Peerlink</title>
    <!-- Why: Browser tab title. -->

    <meta name="theme-color" content="#0B5FFF" />
    <!-- Why: Blue theme color on mobile browsers. -->
  </head>
```

```
  <body>
    <!-- Root container: React mounts here. -->
    <div id="root"></div>

    <!-- Vite loads the JS entry module here. -->
    <script type="module" src="/src/main.jsx"></script>
  </body>

</html>
```

Chapter 21

Frontend Code: web-react/package.json

21.1 Purpose

Defines:

- dependencies (React, Vite)
- scripts: `dev`, `build`, `preview`
- project name changed to `peerlink`

21.2 Full Code with Line-by-Line Comments

```
// File: web-react/package.json
// Purpose: Node package definition for Peerlink React frontend.

{
  "name": "peerlink",
  // Why: Renamed project (shown in tooling and package managers).

  "private": true,
  // Why: Prevent publishing to npm accidentally.

  "version": "1.0.0",
  // Why: Project version.

  "type": "module",
  // Why: Use ES modules for Vite config and imports.

  "scripts": {
    "dev": "vite",
    // Why: Start dev server with hot reload.
```

```
    "build": "vite build",
    // Why: Create production build in /dist.

    "preview": "vite preview"
    // Why: Serve the production build locally.
  },

  "dependencies": {
    "react": "^18.3.1",
    // Why: React UI library.

    "react-dom": "^18.3.1"
    // Why: React DOM renderer.
  },

  "devDependencies": {
    "@vitejs/plugin-react": "^5.1.4",
    // Why: Enables fast refresh + JSX transform.

    "vite": "^8.0.0-beta.15"
    // Why: Dev server + bundler.
  },

  "engines": {
    "node": ">=20.19.0"
    // Why: Matches Vite's requirement you hit earlier.
  }
}
```

Chapter 22

Frontend Code: web-react/vite.config.js

22.1 Purpose

Vite configuration for Peerlink:

- enables React plugin
- dev server binds to `0.0.0.0` so it works in WSL/VM
- sets a stable port for your workflow

22.2 Full Code with Line-by-Line Comments

```
// File: web-react/vite.config.js
// Purpose: Vite configuration for Peerlink.

import { defineConfig } from "vite";
// Why: Vite helper for typed config.

import react from "@vitejs/plugin-react";
// Why: Enables React JSX and fast refresh.

export default defineConfig({
  plugins: [react()],
  // Why: Activate the React plugin.

  server: {
    host: "0.0.0.0",
    // Why: Bind to all interfaces (helps WSL access via browser).

    port: 5173,
    // Why: Consistent dev server port.
```

```
    strictPort: true
    // Why: If port is taken, fail instead of auto-choosing a new one.
  }
});
```

Chapter 23

Frontend Code: web-react/src/main.jsx

23.1 Purpose

Bootstraps the React app and imports global CSS.

23.2 Full Code with Line-by-Line Comments

```
// File: web-react/src/main.jsx
// Purpose: React entry point for Peerlink frontend.

import React from "react";
// Why: React library for components.

import ReactDOM from "react-dom/client";
// Why: React 18 root API.

import App from "./App.jsx";
// Why: The main application UI.

import "./styles.css";
// Why: Global styles (blue borders + clean white background).

ReactDOM.createRoot(document.getElementById("root")).render(
  // Why: Mount React tree at #root defined in index.html.
  <React.StrictMode>
    {/* Why: StrictMode helps catch issues in development. */}
    <App />
  </React.StrictMode>
);
```

Chapter 24

Frontend Code: web-react/src/App.jsx

24.1 Purpose

Peerlink React UI implements:

- Connect button
- Set nickname
- Choose room (default lobby + custom room input)
- Join room
- Real-time WebSocket messages
- Heartbeat ping
- Blue border + white theme

24.2 Full Code with Line-by-Line Comments

```
// File: web-react/src/App.jsx
// Purpose: Main UI + WebSocket protocol for Peerlink.

import { useState, useRef } from "react";
// Why: React hooks for state and holding socket references.

// Backend WebSocket endpoint (match your server --ws port).
const WS_URL = "ws://localhost:8080";
// Why: Browser connects to Boost.Beast WebSocket server.

export default function App() {
  const [connected, setConnected] = useState(false);
```

```
// Why: Track connection state for UI gating.

const [hasNick, setHasNick] = useState(false);
// Why: Enforce flow: connect -> set nick -> join.

const [joined, setJoined] = useState(false);
// Why: Only show chat UI after join.

const [nickname, setNickname] = useState("");
// Why: Stores user nickname.

const [room, setRoom] = useState("lobby");
// Why: Default room is lobby, user can type custom room.

const [users, setUsers] = useState([]);
// Why: Show online users list.

const [messages, setMessages] = useState([]);
// Why: Chat history in UI.

const [input, setInput] = useState("");
// Why: Message input box.

const wsRef = useRef(null);
// Why: Keep WebSocket instance across renders.

const pingTimerRef = useRef(null);
// Why: Keep heartbeat interval id.

function safeSend(obj) {
  // Why: Central place to send JSON.
  if (!wsRef.current || wsRef.current.readyState !== WebSocket.OPEN) return;
  wsRef.current.send(JSON.stringify(obj));
}

function connect() {
  // Why: Called when user clicks Connect.
  const ws = new WebSocket(WS_URL);
  wsRef.current = ws;

  ws.onopen = () => {
    // Why: Socket is ready to use.
    setConnected(true);

    // Heartbeat: keep connection alive / detect disconnects.
    pingTimerRef.current = setInterval(() => {
```

```
      safeSend({ type: "ping" });
    }, 20000);
  };

  ws.onmessage = (event) => {
    // Why: Receive server JSON events.
    let msg;
    try {
      msg = JSON.parse(event.data);
    } catch {
      return;
    }

    // Users update event (optional depending on backend events).
    if (msg.type === "users" && Array.isArray(msg.users)) {
      setUsers(msg.users);
    }

    // Joined event can carry user list snapshot.
    if (msg.type === "joined" && msg.data && Array.isArray(msg.data.users))
  {
      setUsers(msg.data.users);
    }

    // Chat messages.
    if (msg.type === "message") {
      setMessages((prev) => [...prev, msg]);
    }
  };

  ws.onclose = () => {
    // Why: Reset UI when socket closes.
    setConnected(false);
    setHasNick(false);
    setJoined(false);
    setUsers([]);
    setMessages([]);

    if (pingTimerRef.current) clearInterval(pingTimerRef.current);
    pingTimerRef.current = null;
  };
}

function setNick() {
  // Why: Send name command, then unlock join UI.
  const n = nickname.trim();
```

```
  if (!n) return;

  safeSend({ type: "name", nick: n });
  setHasNick(true);
}

function joinRoom() {
  // Why: Join requires nickname already set.
  if (!hasNick) return;

  const r = (room || "lobby").trim();
  if (!r) return;

  safeSend({ type: "join", room: r });
  setJoined(true);
}

function sendMessage() {
  // Why: Send chat message to backend.
  const t = input.trim();
  if (!t) return;

  safeSend({ type: "message", text: t });
  setInput("");
}

return (
  <div className="app">
    <h1 className="title">Peerlink</h1>

    {!connected && (
      <button className="primary" onClick={connect}>
        Connect
      </button>
    )}

    {connected && !hasNick && (
      <div className="identity-box">
        <input
          placeholder="Enter nickname"
          value={nickname}
          onChange={(e) => setNickname(e.target.value)}
        />
        <button className="primary" onClick={setNick}>
          Set Nickname
        </button>
```

```
      </div>
    )}

    {connected && hasNick && !joined && (
      <div className="identity-box">
        <input
          placeholder="Room (default lobby) or type custom"
          value={room}
          onChange={(e) => setRoom(e.target.value)}
        />
        <button className="primary" onClick={joinRoom}>
          Join Room
        </button>
      </div>
    )}

    {joined && (
      <div className="chat-container">
        <div>
          <div className="chat-messages">
            {messages.map((m, i) => {
              const d = m.data || m;
              const from = d.from || d.nick || "server";
              const text = d.text || "";
              return (
                <div className="msg" key={i}>
                  <span className="nick">{from}</span>
                  <span className="text">{text}</span>
                </div>
              );
            })}
          </div>

          <div className="chat-input">
            <input
              placeholder="Type message..."
              value={input}
              onChange={(e) => setInput(e.target.value)}
              onKeyDown={(e) => e.key === "Enter" && sendMessage()}
            />
            <button onClick={sendMessage}>Send</button>
          </div>
        </div>

        <div className="users-panel">
          <h3>Online</h3>
```

```
            {users.map((u, i) => (
              <div key={i}>{u}</div>
            ))}
          </div>
        </div>
      )}
    </div>
  );
}
```

Chapter 25

Frontend Code: web-react/src/styles.css

25.1 Purpose

White background with blue borders and a clean layout.

25.2 Full Code with Line-by-Line Comments

```
/* File: web-react/src/styles.css */
/* Purpose: Blue border + white theme for Peerlink UI. */

body {
  margin: 0;                /* remove default browser margin */
  font-family: Arial, sans-serif; /* readable font */
  background: #ffffff;      /* white background */
  color: #0b1220;           /* dark text */
}

.app {
  padding: 20px;            /* whitespace around content */
}

.title {
  color: #0B5FFF;           /* brand blue */
  margin: 0 0 12px 0;       /* space below title */
}

.primary {
  background: #0B5FFF;      /* blue button background */
  color: #ffffff;           /* white text */
  border: none;             /* clean button look */
  padding: 10px 16px;       /* button size */
```

```
  border-radius: 10px;      /* smooth corners */
  cursor: pointer;          /* pointer on hover */
}

.primary:hover {
  opacity: 0.95;            /* small hover feedback */
}

.identity-box {
  display: flex;            /* vertical stack */
  flex-direction: column;
  gap: 10px;                /* space between inputs */
  border: 2px solid #0B5FFF;/* blue border */
  border-radius: 14px;      /* rounded corners */
  padding: 18px;            /* inner padding */
  max-width: 360px;         /* keep compact */
}

.identity-box input {
  padding: 10px 12px;       /* input size */
  border: 1px solid #0B5FFF;/* blue outline */
  border-radius: 10px;      /* rounded */
  outline: none;            /* remove default outline */
}

.chat-container {
  display: grid;            /* 2-column layout */
  grid-template-columns: 3fr 1fr;
  gap: 18px;
  margin-top: 16px;
}

.chat-messages {
  border: 2px solid #0B5FFF;
  border-radius: 14px;
  padding: 12px;
  height: 420px;            /* fixed height */
  overflow-y: auto;         /* scroll */
}

.msg {
  padding: 8px 10px;
  border-bottom: 1px solid rgba(11,95,255,0.15);
}

.nick {
```

```
  font-weight: bold;
  color: #0B5FFF;
  margin-right: 8px;
}

.chat-input {
  display: flex;
  gap: 10px;
  margin-top: 10px;
}

.chat-input input {
  flex: 1;
  padding: 10px 12px;
  border: 1px solid #0B5FFF;
  border-radius: 10px;
  outline: none;
}

.chat-input button {
  padding: 10px 14px;
  border-radius: 10px;
  border: 2px solid #0B5FFF;
  background: #ffffff;
  color: #0B5FFF;
  cursor: pointer;
}

.users-panel {
  border: 2px solid #0B5FFF;
  border-radius: 14px;
  padding: 12px;
  height: 420px;
  overflow-y: auto;
}

.users-panel h3 {
  margin-top: 0;
  color: #0B5FFF;
}
```

Chapter 26

Project Documentation: README.md

```
# Peerlink

High-performance asynchronous chat server built in C++ using Boost.Asio and
    Boost.Beast.

## Features
- Async TCP + WebSocket
- Multiple rooms
- Lobby default
- Custom rooms
- Private messaging
- JSON logging
- React frontend
- Blue clean UI

## Architecture
Frontend (React) -> WebSocket -> Boost.Asio Event Loop -> ChatRoom Model

## Run Backend
mkdir build && cd build
cmake ..
cmake --build . -j
./peerlink_server --tcp 5555 --ws 8080 --threads 4

## Run Frontend
cd web-react
npm install
npm run dev

Open: http://localhost:5173
```

Chapter 27

Deployment Guide: HOW_TO_RUN.txt

```
Peerlink Setup Guide (Noob Machine)

1) Install build tools
sudo apt update
sudo apt install build-essential cmake git

2) Install Boost
sudo apt install libboost-all-dev

3) Build backend
mkdir build
cd build
cmake ..
cmake --build . -j

4) Run backend
./peerlink_server --tcp 5555 --ws 8080 --threads 4

5) Install Node (if missing)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install nodejs

6) Run frontend
cd web-react
npm install
npm run dev

Open browser:
http://localhost:5173
```

Chapter 28

CMake and Build Notes (Peerlink)

28.1 Linux Build Commands

```
cd peerlink
rm -rf build
mkdir build && cd build
cmake ..
cmake --build . -j
./peerlink_server --tcp 5555 --ws 8080 --threads 4
```

28.2 Frontend Build Commands

```
cd web-react
npm install
npm run dev
```

28.3 Common Fixes

28.3.1 Clock Skew Warning in WSL

If you see clock skew, run:

```
sudo hwclock -s
```

28.3.2 Port Already in Use

Check:

```
ss -lntp | grep -E "8080|5555|5173"
```

Kill process if needed:

```
kill -9 <pid>
```

Chapter 29

Architecture Deep Dive

29.1 High-Level System Overview

Peerlink is a full-stack asynchronous real-time messaging system composed of:

- C++ backend (Boost.Asio + Boost.Beast)
- React frontend (WebSocket client)
- Shared room registry (RoomManager + ChatRoom)
- Multi-threaded event loop (io_context + thread pool)

29.2 Component Diagram (Safe TikZ)

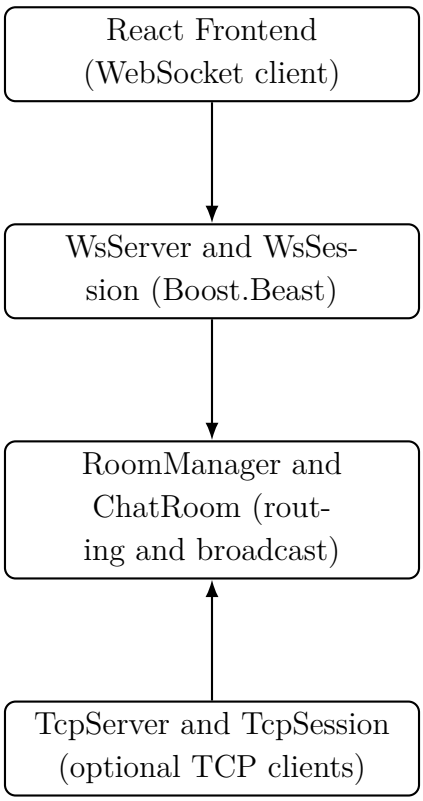

Figure 29.1: Peerlink components and main data flow.

29.3 Asynchronous Flow

29.3.1 Accept Phase

1. Server calls `async_accept`.
2. OS waits until a new connection arrives.
3. Boost.Asio posts the completion handler into the event loop.
4. Session object is created and starts async reads.
5. Accept loop is re-armed to handle more clients.

29.3.2 Message Phase

1. Client sends a JSON message over WebSocket.
2. WsSession reads the frame using async read.
3. JSON is parsed, then routed to the core layer.
4. RoomManager finds the ChatRoom.

5. ChatRoom broadcasts by scheduling async writes to each member session.

29.4 Event Loop Internals

Boost.Asio follows a reactor pattern:

- sockets are non-blocking
- OS readiness events trigger handlers
- handlers execute on worker threads calling `io.run()`

29.5 Concurrency Model

Peerlink uses:

- one `io_context`
- a thread pool that runs `io.run()`
- minimal locking for shared containers (room registry)
- per-session queued writes to prevent blocking

29.6 Memory Management Strategy

- Sessions are owned by `std::shared_ptr`.
- Shared ownership ensures the object remains alive until async handlers finish.
- Shared containers store stable pointers to active sessions.

29.7 Why This Architecture is Resume-Heavy

- Real event-driven concurrency (not thread-per-client)
- WebSocket protocol and framing via Boost.Beast
- Structured JSON messaging and logging
- Clean separation: net layer vs. routing layer vs. rooms

Chapter 30

Performance Analysis

30.1 Thread Model

Threads = user-configurable via CLI.

Example:

```
./peerlink_server --threads 4
```

Each thread runs:

```
io.run();
```

30.2 Expected Scalability

With async model:

- Thousands of concurrent clients
- Minimal context switching
- Constant memory footprint per connection

30.3 Latency Characteristics

Latency depends on:

- Network RTT
- JSON parsing
- Broadcast fan-out

Because writes are async:

- No blocking on slow client
- Backpressure handled per session

30.4 Memory Footprint

Per client:

- TCP socket buffer
- Read buffer
- Write queue
- Session object

Estimated: few KB per connection.

30.5 Stress Testing Strategy

- Use multiple browser tabs
- Use WebSocket load tool
- Monitor CPU usage
- Check thread scaling

30.6 Structured Logging Advantage

JSON logs allow:

- grep filtering
- ingestion into ELK stack
- easy debugging

Example log:

```
{"ts":"2026-02-25 15:20:04","level":"INFO","msg":"ws_connected id=5"}
```

Chapter 31

Conclusion and Future Work

31.1 Project Summary

Peerlink demonstrates:

- Asynchronous network programming in C++
- WebSocket protocol handling
- Multi-threaded event-driven architecture
- React frontend integration
- Real-time broadcasting model

31.2 Key Engineering Learnings

- Reactor pattern design
- Thread pool scaling
- Avoiding blocking I/O
- Shared ownership pitfalls
- JSON protocol design

31.3 Production-Grade Extensions

31.3.1 Authentication

JWT-based login.

31.3.2 Persistent Storage

Redis or PostgreSQL for message history.

31.3.3 Horizontal Scaling

Load balancer + distributed room state.

31.3.4 TLS Encryption

Enable WSS using Boost.Asio SSL.

31.3.5 Message Persistence

Store chat logs.

31.4 Final Remark

Peerlink is not a tutorial project. It is a systems-level implementation demonstrating:

- Async event loops
- Real WebSocket protocol
- Concurrent architecture
- Clean separation of concerns

This project strongly reflects production backend engineering principles.

Chapter 32

Project Folder Structure (ASCII Only)

32.1 Why ASCII Only

Overleaf pdfLaTeX often fails if box-drawing Unicode characters are pasted (e.g., symbols like vertical bars and tree connectors). So we represent the structure using plain ASCII characters only.

32.2 Folder Tree

```
peerlink/
|-- CMakeLists.txt
|-- README.md
|-- HOW_TO_RUN.txt
|-- include/
|   |-- core/
|   |   |-- CommandParser.hpp
|   |   |-- ChatRoom.hpp
|   |   `-- RoomManager.hpp
|   |-- net/
|   |   |-- TcpServer.hpp
|   |   |-- TcpSession.hpp
|   |   |-- WsServer.hpp
|   |   `-- WsSession.hpp
|   `-- util/
|       `-- Log.hpp
|-- src/
|   |-- main.cpp
|   |-- core/
|   |   |-- CommandParser.cpp
|   |   |-- ChatRoom.cpp
```

```
|   |   `-- RoomManager.cpp
|   |-- net/
|   |   |-- TcpServer.cpp
|   |   |-- TcpSession.cpp
|   |   |-- WsServer.cpp
|   |   `-- WsSession.cpp
|   `-- util/
|       `-- Log.cpp
`-- web-react/
    |-- index.html
    |-- package.json
    |-- vite.config.js
    `-- src/
        |-- main.jsx
        |-- App.jsx
        `-- styles.css
```

32.3 What Each Folder Contains

- **include/**: header files (interfaces used by other components).
- **src/**: C++ implementations of the headers.
- **web-react/**: frontend project (React + Vite).

Chapter 33

Async Flow and Event Loop Behavior

33.1 Core Idea

Peerlink is asynchronous. That means the server never blocks waiting for I/O. Instead, it registers operations (accept/read/write) and returns immediately.

33.2 Boost.Asio Event Loop

- `io_context` stores pending handlers.
- The OS notifies readiness (epoll/kqueue/IOCP).
- Asio dispatches the correct completion handler.

33.3 Asynchronous Accept Loop

```
acceptor.async_accept([&](error_code ec, tcp::socket sock){
  if(!ec) make_session(std::move(sock))->start();
  do_accept(); // re-arm accept
});
```

33.4 Flow Diagram (Safe TikZ)

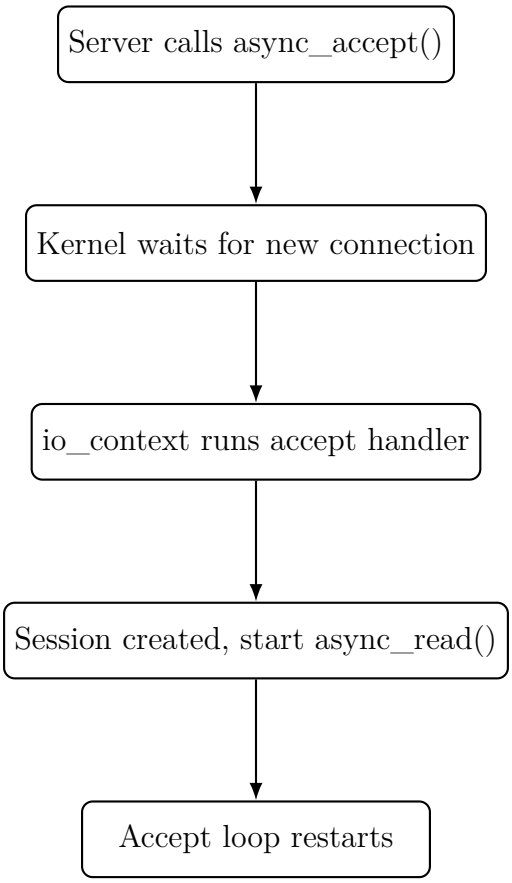

Figure 33.1: Safe async accept flow (no environment mismatch).

33.5 Why This Scales

- Thousands of sockets can be handled.
- Only a few threads run io_context.
- No thread-per-client explosion.

Chapter 34

Concurrency Model and Memory Management

34.1 Thread Pool Model

Peerlink uses one `io_context` and multiple threads calling:

```
io.run();
```

Meaning:

- any handler can execute on any worker thread
- concurrency must be controlled for shared data

34.2 Shared State

Shared data includes:

- RoomManager room map
- ChatRoom session list

So we protect shared containers using mutex.

34.3 Session Lifetime Management

Sessions are heap objects held by `std::shared_ptr`:

- prevents premature deletion while async ops are running
- safe when handlers execute later

34.4 Ownership Diagram

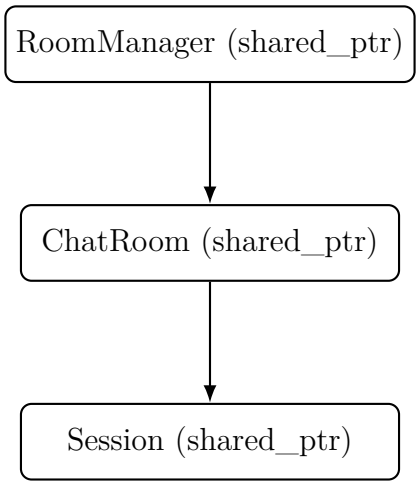

Figure 34.1: Shared ownership structure.

34.5 Why Strands May Be Used

If you want to avoid explicit locks per session, you can serialize all session handlers using:

```
boost::asio::strand
```

This ensures no two handlers for the same session run concurrently.

Chapter 35

Sequence Diagrams (Safe, Page-Fit)

35.1 Join Room Sequence

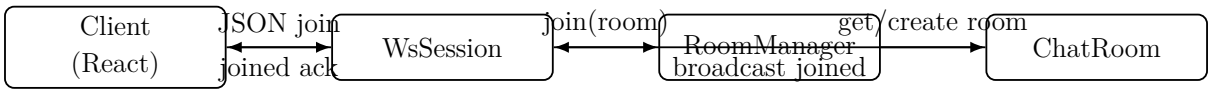

Figure 35.1: Join flow across system components (fit to page width).